AI-Lingo — Project Planning, Literature Review & Requirements Gathering

1. Project Planning & Management

1.1 Project Proposal

Project Proposal: AI-Lingo

Project Proposal: AI-Lingo

Project Overview

AI-Lingo is an AI-powered language-learning mobile application developed using Flutter and Dart. The application aims to help users improve their language skills through personalized learning experiences and AI-generated contextual conversations.

Unlike traditional learning methods that mainly rely on predefined vocabulary lists and exercises, AI-Lingo allows users to select their target language, proficiency level, interests, and learning goals. Based on these preferences, the application provides relevant learning scenarios and interactive conversations that simulate real-life situations.

The application also includes a Vocabulary section where users can save and manage useful words collected during learning sessions. User preferences and vocabulary can be stored locally to provide a more convenient learning experience.

Problem Statement

Many language learners struggle to practice a new language in realistic situations. Traditional learning methods may provide limited opportunities for interactive conversation and personalized content that matches individual interests and proficiency levels.

AI-Lingo addresses these challenges by combining mobile application development, artificial intelligence, personalized learning scenarios, contextual conversations, and vocabulary management.

Project Aim

To develop a user-friendly, cross-platform mobile application that supports personalized language learning through AI-generated contextual conversations and vocabulary management.

Project Objectives

1. Develop a cross-platform mobile application using Flutter and Dart.
2. Allow users to register, log in, and configure their learning preferences.
3. Personalize learning scenarios according to the user's language level, interests, and goals.
4. Provide AI-generated conversational practice based on realistic situations.
5. Allow users to save useful vocabulary collected during conversations.
6. Store relevant user preferences and saved vocabulary locally where appropriate.
7. Apply structured software architecture and state management principles.
8. Protect user information and handle API errors securely.
9. Test the application's core features for functionality, usability, and reliability.

Project Scope

In Scope

- Splash and onboarding screens.
- User registration and login.
- Learning personalization.
- Home screen and learning scenarios.
- AI-powered conversation screen.
- Vocabulary management.
- Profile and settings.
- Local data persistence for supported information.
- Error handling and application testing.

Out of Scope for the Initial Version

- A dedicated progress dashboard.
- A separate flashcards and spaced-repetition module.
- Advanced placement testing.
- User-created custom scenarios.
- Social learning and multiplayer features.

Target Users

- Beginners learning a new language.
- Intermediate learners who need additional conversation practice.
- Students who want to improve their vocabulary and communication confidence.
- Self-directed learners who prefer personalized learning experiences.

Expected Outcome

The expected outcome is a functional Flutter application that enables users to configure their learning preferences, explore relevant scenarios, practice language through AI-generated conversations, save useful vocabulary, and manage supported profile settings.

1.2 Project Plan

The project is planned over an estimated eight-week period. This timeline is a proposal and should be adjusted according to the actual academic deadline and team progress.

Proposed Gantt Chart

AI-Lingo — 8-Week Timeline

Milestones and Deliverables

Milestone	Target Week	Deliverable
M1: Requirements approved	Week 1	Requirements and scope document
M2: UI/UX completed	Week 2	Screen designs and navigation flow
M3: Core architecture ready	Week 3	Flutter structure and state management setup

Milestone	Target Week	Deliverable
M4: Core features implemented	Week 5	Authentication, personalization, home, and learning scenarios
M5: AI and vocabulary integrated	Week 6	Conversation and vocabulary functionality
M6: Testing completed	Week 7	Test cases, test results, and bug report
M7: Final submission	Week 8	Application, final report, and presentation

Resource Allocation

Resource	Purpose
Flutter and Dart	Cross-platform mobile application development
Cubit/BLoC	Application state management
AI API and prompt engineering	Contextual conversation generation
Hive or another local database	Local persistence, if selected
Git and GitHub	Version control and team collaboration
Testing tools	Functional, integration, and usability testing
Team members	Development, research, testing, integration, and documentation

The final report should list the technologies actually used in the implemented application rather than proposed technologies that have not been adopted.

1.3 Task Assignment & Roles

The project team consists of five members. Each member is assigned a primary set of features, with collaboration between developers wherever features share data or functionality.

1. Nada Mohamed Elsaadani — Team Leader

Assigned Features: Splash, Onboarding, Authentication, and Personalization.

Responsibilities:

- Lead project planning, task distribution, and progress tracking.
- Coordinate communication and integration between team members.
- Implement the Splash and Onboarding screens.
- Develop Login and Registration screens and connect them to the configured authentication service.
- Implement the Personalization screen.
- Collect and validate learning preferences, including target language, proficiency level, interests, and learning goals.
- Ensure that learning preferences are stored and made available to the Learning and AI Conversation features.
- Coordinate testing, integration, and final project delivery.

2. Naglaa Mohamed Shaaban

Assigned Feature: Vocabulary Management.

Responsibilities:

- Design and implement the Vocabulary screen.
- Allow users to save useful words collected during AI conversations.
- Implement vocabulary listing, viewing, and supported editing or deletion operations.
- Integrate local persistence to retain saved vocabulary across application restarts.
- Coordinate with the AI Conversation developer to receive vocabulary entries in an agreed data format.
- Test vocabulary saving, retrieval, and data consistency.

3. Reem Hany Mustafa

Assigned Feature: AI Conversation.

Responsibilities:

- Design and implement the AI Conversation screen.
- Integrate the application with the selected AI service through the configured API or backend.
- Prepare prompts using the learner's target language, proficiency level, interests, and selected scenario.
- Implement contextual conversation flow and display AI-generated responses.
- Handle loading states, API failures, timeouts, and invalid responses.
- Support the identification or extraction of useful vocabulary from conversations.
- Test the relevance and consistency of generated conversations.

4. Mai Mohamed Abdo

Assigned Feature: Learning / Scenarios.

Responsibilities:

- Design and implement the Learning screen and scenario-browsing interface.
- Display available learning scenarios in an organized format.
- Organize scenarios by categories, interests, and difficulty levels where supported.
- Connect selected scenarios to the AI Conversation feature.
- Ensure that learner preferences are considered when displaying or selecting scenarios.
- Coordinate with the Team Leader and AI Conversation developer to define the data passed between screens.
- Test scenario selection, navigation, and integration.

5. Mohamed Reda Mossa

Assigned Features: Profile and Settings.

Responsibilities:

- Design and implement the Profile screen.
- Develop the Settings interface for supported user preferences.
- Display and manage available user information.

- Implement profile editing and saving where supported.
- Coordinate with the Team Leader to access authentication-related user information and shared learning preferences.
- Implement logout functionality if supported by the authentication system.
- Test profile updates, settings persistence, and navigation.

Team Collaboration and Integration

All team members are responsible for testing their assigned features, maintaining code quality, and communicating interface or dependency changes.

The Team Leader coordinates integration and monitors project milestones. The team should agree on shared data models, navigation contracts, repository interfaces, and state-management conventions before combining feature branches.

Git/GitHub will be used for version control and collaboration. Integration testing will be performed after features are combined.

The Home screen is a shared integration point. One developer should be explicitly assigned responsibility for implementing its UI, while the relevant feature developers coordinate its navigation and data connections. This avoids duplicated work and conflicts during integration.

1.4 Risk Assessment & Mitigation Plan

Risk	Impact	Probability	Mitigation Strategy
AI API becomes unavailable	High	Medium	Implement timeouts, error handling, and retry options
AI responses are inaccurate or irrelevant	High	Medium	Improve prompts, validate responses, and test different scenarios
API key exposure in the mobile app	High	Medium	Store private credentials on a secure backend
Unstable internet connection	Medium	High	Display connection errors and keep saved vocabulary accessible locally
Local data loss or corruption	High	Low–Medium	Validate stored data and test persistence and recovery
Team integration conflicts	Medium	Medium	Define interfaces, use Git branches, and review changes before merging
Scope exceeds the available time	High	Medium	Prioritize core features and defer optional functionality
Late discovery of bugs	High	Medium	Test features incrementally and reserve time for bug fixing

1.5 KPIs (Key Performance Indicators)

The following indicators are proposed for evaluating the project's success. Target values are objectives for testing, not measured results.

KPI	Target	Measurement Method
AI response time	95% of responses within 5 seconds, where feasible	Measure request-to-response duration
Crash-free sessions	At least 99% during the test period	Review crash logs and test sessions
Core test pass rate	At least 95% of executed test cases	Passed tests ÷ total executed tests

KPI	Target	Measurement Method
Vocabulary persistence	100% of verified save/retrieve test cases pass	Restart the app and verify saved entries
Key-task completion rate	At least 90% of test users complete key tasks	Conduct representative usability tests
AI request success rate	At least 95% under defined test conditions	Successful requests ÷ total requests
User satisfaction	Average 4/5 or higher	Post-test questionnaire
On-time-milestone completion	At least 90%	Compare planned and actual completion dates

System uptime is primarily relevant if the project includes a continuously available backend. For a mobile prototype that depends on a third-party AI service, API request success and application reliability should be measured separately.

2. Literature Review

2.1 Literature Review Overview

Literature Review: AI-Powered Language Learning

Literature Review: AI-Powered Language Learning

Introduction

Language-learning applications have evolved from static vocabulary lists and grammar exercises into interactive digital platforms that support personalized learning and conversational practice. Mobile applications provide learners with flexible access to educational content, while artificial intelligence can generate context-specific dialogues and adapt learning materials to individual needs.

AI-Lingo builds on these concepts by combining mobile learning, personalized content generation, contextual conversations, and vocabulary collection within a single Flutter application.

Key Areas Reviewed

1. Mobile-Assisted Language Learning (MALL)

Mobile-Assisted Language Learning uses mobile devices and applications to support language acquisition. It enables learners to access learning materials at different times and locations. AI-Lingo follows this approach by providing a mobile interface for language practice and vocabulary management.

2. Personalized Learning

Personalized learning adapts educational content to a learner's proficiency, interests, and goals. AI-Lingo collects these preferences during the personalization stage and uses them to guide scenario selection and AI prompts.

3. AI-Based Conversational Practice

Large language models can generate dialogues and simulate real-life communication situations. In AI-Lingo, an AI service can generate contextual conversations based on the selected language, learner level, and scenario. Generated content must be evaluated for relevance, correctness, and suitability.

4. Vocabulary Acquisition

Vocabulary learning benefits from repeated exposure to words in meaningful contexts. AI-Lingo supports this process by allowing users to collect useful words from conversations and access them later through the Vocabulary section.

5. Mobile Application Architecture

Structured architectures, state management, and repository patterns help separate the user interface from business logic and data access. AI-Lingo uses Flutter and can apply Cubit/BLoC and repository-based organization to improve maintainability and testability.

Research Gap and Project Contribution

Traditional mobile language-learning tools may emphasize predefined lessons or exercises, while general-purpose AI chat tools may not provide a focused vocabulary workflow or structured learner preferences. AI-Lingo aims to combine personalized scenario generation, contextual conversation, and vocabulary collection in one mobile experience.

Conclusion

The reviewed concepts support the design of AI-Lingo as a personalized language-learning prototype. Its effectiveness should be evaluated through functional testing, AI response-quality assessment, and usability testing. This review should be supplemented with verified academic papers and properly formatted references before final submission.

2.2 Feedback & Evaluation — Lecturer's Assessment

The lecturer's assessment should be completed after the project has been reviewed. The following template can be used to record the actual feedback.

Lecturer's Feedback and Evaluation

Lecturer's Feedback and Evaluation

Project-Name:AI-Lingo
Evaluator:
Evaluation Date:

Evaluation Areas

- Clarity of project objectives and problem definition.
- Relevance of the proposed AI-based solution.
- Quality of the user interface and user experience.
- Software architecture and code organization.
- AI integration and quality of generated conversations.
- Data persistence and security considerations.
- Testing quality and documentation.

Lecturer's Feedback

Strengths:

Areas for Improvement:

Required Changes:

Overall Evaluation:

2.3 Suggested Improvements

Suggested Improvements for AI-Lingo

Suggested Improvements for AI-Lingo

The following improvements may be considered during future development:

1. **Improve AI response quality:** Refine prompts and validate generated conversations for relevance, language level, and grammatical accuracy.
2. **Enhance personalization:** Use learner preferences and previously completed scenarios to select more relevant learning content.
3. **Strengthen vocabulary support:** Add definitions, example sentences, pronunciation, and a review mechanism in a future release.
4. **Improve offline accessibility:** Ensure that saved vocabulary and essential profile information remain accessible without an internet connection.
5. **Strengthen security:** Store API credentials on a secure backend, protect sensitive user data, and apply appropriate authentication and authorization controls.
6. **Expand testing:** Add unit tests, integration tests, API error tests, and usability tests with representative learners.
7. **Improve accessibility:** Support readable typography, sufficient color contrast, clear feedback messages, and accessible controls.
8. **Evaluate educational effectiveness:** Conduct user testing to measure whether contextual conversations help learners improve confidence and vocabulary retention.

These improvements should be prioritized according to available time, implementation complexity, and feedback from actual users and the lecturer.

2.4 Final Grading Criteria

The following is a proposed 100-mark assessment rubric. It is not an official grading breakdown unless approved by the lecturer.

Assessment Component	Marks	Evaluation Criteria
Project planning and management	10	Proposal, timeline, task allocation, risks, and KPIs
Literature review and research	15	Relevant research, critical analysis, and references
Requirements analysis	15	Stakeholders, user stories, use cases, and requirements

Assessment Component	Marks	Evaluation Criteria
UI/UX and implementation	25	Functionality, usability, architecture, and code quality
Testing and evaluation	15	Test coverage, results, bug handling, and evaluation
Documentation	10	Structure, clarity, consistency, and technical details
Final presentation and demonstration	10	Explanation, demonstration, and responses to questions
Total	100	

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder Analysis — AI-Lingo

Stakeholder Analysis — AI-Lingo

Stakeholder analysis identifies the people and organizations that influence the project or benefit from its outcomes.

Stakeholder	Role / Interest	Main Needs
Language learners	Primary end users	Easy navigation, relevant conversations, suitable difficulty, and saved vocabulary
Project team	Developers and maintainers	Clear requirements, maintainable code, defined responsibilities, and manageable scope
Lecturer/ Project Supervisor	Academic evaluator	Clear objectives, technical justification, working features, and documented testing
AI service provider	External technical dependency	Valid API requests, compliance with service limits, and secure integration
Backend administrator, if applicable	Maintains the server and API integration	Secure credential management, monitoring, and reliable API handling
Testing participants	Evaluate the prototype	Simple tasks, understandable instructions, and a way to provide feedback

Stakeholder Priorities

The primary priority is the learner's experience. The application should make it straightforward to configure learning preferences, start a relevant conversation, and save useful vocabulary.

The project team and lecturer require a clear and testable implementation. External AI services introduce additional dependencies, so the application should handle network errors and service limitations gracefully.

3.2 User Stories

User stories describe the system from the end user's perspective and help the team prioritize development.

ID	User Story	Priority
US-01	As a user, I want to view an onboarding introduction so that I understand the application's purpose.	Medium
US-02	As a user, I want to register or log in so that I can access my learning experience.	High
US-03	As a learner, I want to choose my target language and proficiency level so that learning content matches my needs.	High
US-04	As a learner, I want to specify my interests and goals so that scenarios are relevant to me.	High
US-05	As a learner, I want to browse learning scenarios so that I can choose a situation to practice.	High
US-06	As a learner, I want to interact with an AI-generated conversation so that I can practice realistic language use.	High
US-07	As a learner, I want to save useful words from conversations so that I can revisit them later.	High
US-08	As a learner, I want to view my saved vocabulary so that I can review the words I collected.	High
US-09	As a user, I want to manage my profile and preferences so that my learning settings remain relevant.	Medium
US-10	As a user, I want clear error messages when a request fails so that I understand what happened and what to do next.	High

3.3 Use Cases

Use Case Specifications — AI-Lingo

UC-01: User Authentication

Primary Actor: User

Preconditions: The application is installed, and a network connection is available if online authentication is required.

Main Flow:

1. The user opens the application.
2. The user chooses Login or Register.
3. The user enters the required credentials.
4. The application validates the input.
5. The configured authentication service processes the request.
6. The application displays the result and navigates the user to the appropriate screen.

Alternative Flows:

- Invalid credentials: Display an understandable error message.
- Missing input: Ask the user to complete required fields.
- Network failure: Display a connection error and allow retrying.

Postcondition: The user is authenticated, or the application explains why authentication failed.

UC-02: Personalize Learning

Primary Actor: Learner

Preconditions: The user has opened the Personalization screen.

Main Flow:

1. The user selects a target language.
2. The user selects a proficiency level.
3. The user chooses interests and learning goals.
4. The application validates the selections.
5. The preferences are saved.
6. The application opens the Home screen.

Alternative Flow: If required selections are missing, the application requests the missing information.

Postcondition: Valid learning preferences are available to the scenario-selection and conversation features.

UC-03: Start an AI Conversation

Primary Actor: Learner

Supporting System: AI service and backend, if used.

Preconditions: The learner has selected a scenario, and the AI service is available.

Main Flow:

1. The learner browses available scenarios.
2. The learner selects a scenario.
3. The application prepares a prompt using the learner's language, level, interests, and selected context.
4. The application sends the request through the configured AI integration.
5. The AI service returns a response.
6. The application displays the conversation.
7. The learner continues interacting with the AI.

Alternative Flows:

- API timeout or network failure: Display an error and allow retrying.
- Invalid or incomplete AI response: Validate the response and handle it safely.
- API quota exceeded: Inform the user that the service is temporarily unavailable.

Postcondition: The learner can practice the selected scenario or receives a clear explanation when the interaction cannot continue.

UC-04: Save and View Vocabulary

Primary Actor: Learner

Preconditions: The learner has access to a conversation or Vocabulary screen.

Main Flow:

1. The learner identifies a useful word.
2. The learner selects the save action.
3. The application validates the vocabulary entry.
4. The application stores it in local persistence.

5. The learner opens the Vocabulary screen.
6. The application displays saved entries.

Alternative Flows:

- Duplicate word: Prevent duplicate entries or notify the user.
- Storage failure: Display an error and do not report the word as saved.

Postcondition: The vocabulary entry is available for later access if saving succeeds.

UC-05: Manage Profile and Settings

Primary Actor: User

Preconditions: The user has opened Profile/Settings.

Main Flow:

1. The user views available profile information.
2. The user edits supported fields or preferences.
3. The application validates the changes.
4. The application saves the changes.
5. The application confirms the result.

Alternative Flow: If saving fails, the application preserves the previous valid data and informs the user.

Postcondition: Updated information is saved successfully, or the user receives a clear error message.

3.4 Functional Requirements

Functional requirements define what the system must do.

ID	Requirement	Priority
FR-01	The application shall display a Splash screen and onboarding information.	Medium
FR-02	The application shall allow users to register and log in through the configured authentication method.	High
FR-03	The application shall validate required authentication inputs and display errors.	High
FR-04	The application shall allow users to select their target language.	High
FR-05	The application shall allow users to specify their proficiency level, interests, and learning goals.	High
FR-06	The application shall save and retrieve learning preferences.	High
FR-07	The application shall display learning scenarios relevant to the learner's preferences.	High
FR-08	The application shall generate or retrieve contextual conversation content through the configured AI integration.	High
FR-09	The application shall display conversation messages in a readable interface.	High
FR-10	The application shall handle API failures, timeouts, and invalid responses.	High
FR-11	The application shall allow learners to save useful vocabulary entries.	High
FR-12	The application shall display saved vocabulary entries.	High
FR-13	The application shall persist vocabulary locally across application restarts.	High
FR-14	The application shall provide a Profile/Settings screen for supported profile and preference	Medium

ID	Requirement	Priority
	management.	
FR-15	The application shall provide loading, success, empty, and error states where appropriate.	High
FR-16	The application shall allow users to log out if authentication is implemented.	Medium

Requirements dependent on external authentication, an AI service, or a backend should be marked as implemented only after they have been tested in the actual application.

3.5 Non-Functional Requirements

Non-functional requirements define how well the application should operate in terms of performance, security, usability, and reliability.

ID	Category	Requirement	Proposed Acceptance Criteria
NFR-01	Performance	The application shall respond promptly to user actions.	Local navigation and interactions should generally complete within 2 seconds on the test device.
NFR-02	Performance	AI requests shall have a defined timeout.	Requests should time out after a configured limit, such as 30 seconds, and provide a retry option.
NFR-03	Security	API credentials shall not be exposed as secrets in the Flutter client.	Private AI credentials are managed through a secure backend when required.
NFR-04	Security	Authentication and user data shall be handled securely.	Apply secure authentication practices and restrict access to protected data.
NFR-05	Privacy	The application shall collect only the data necessary for its functions.	Explain relevant data collection and avoid sending unnecessary personal information to the AI service.
NFR-06	Usability	The application shall provide a clear and consistent interface.	Representative users should be able to complete core tasks without significant assistance.
NFR-07	Reliability	The application shall handle network and AI service failures gracefully.	Failures should display understandable messages rather than crash the application.
NFR-08	Data Integrity	Saved vocabulary and preferences shall remain consistent.	Saved entries should remain available after an application restart in persistence tests.
NFR-09	Maintainability	The codebase shall separate presentation, business logic, and data access.	Use appropriate architecture patterns, such as Cubit/BLoC and repositories.
NFR-10	Compatibility	The application shall support the agreed target mobile platforms.	Test on the minimum supported Android/iOS versions defined by the team.
NFR-11	Accessibility	Text and interactive elements shall be easy to read and use.	Check text contrast, touch-target sizes, and layout behavior at different text sizes.
NFR-12	Scalability	The system shall support future feature additions without extensive restructuring.	Keep AI integration, persistence, and UI components modular.

These acceptance criteria are proposed targets and should be adjusted according to the actual implementation, test devices, and project constraints.